

```
import pandas as pd
```

```
df = pd.read_csv("diabetes.csv")
```

```
df
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1
..	...	...	...	...	...	...
763	10	101	76	48	180	32.9
764	2	122	70	27	0	36.8
765	5	121	72	23	112	26.2
766	1	126	60	0	0	30.1
767	1	93	70	31	0	30.4

	Pedigree	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1
..	...	...	...
763	0.171	63	0
764	0.340	27	0
765	0.245	30	0
766	0.349	47	1
767	0.315	23	0

```
[768 rows x 9 columns]
```

```
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
0	6	148	72	35	0	33.6

1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	Pedigree	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

df.tail()

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
763	10	101	76	48	180	32.9
764	2	122	70	27	0	36.8
765	5	121	72	23	112	26.2
766	1	126	60	0	0	30.1
767	1	93	70	31	0	30.4

	Pedigree	Age	Outcome
763	0.171	63	0
764	0.340	27	0
765	0.245	30	0
766	0.349	47	1
767	0.315	23	0

df.describe()

	Pregnancies	Glucose	BloodPressure	SkinThickness
Insulin \				
count	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458
std	3.369578	31.972618	19.355807	15.952218
min	0.000000	0.000000	0.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000

```

0.000000
50%      3.000000   117.000000      72.000000      23.000000
30.500000
75%      6.000000   140.250000      80.000000      32.000000
127.250000
max      17.000000   199.000000     122.000000     99.000000
846.000000

```

```

          BMI      Pedigree      Age      Outcome
count  768.000000  768.000000  768.000000  768.000000
mean    31.992578   0.471876   33.240885   0.348958
std      7.884160   0.331329   11.760232   0.476951
min      0.000000   0.078000   21.000000   0.000000
25%     27.300000   0.243750   24.000000   0.000000
50%     32.000000   0.372500   29.000000   0.000000
75%     36.600000   0.626250   41.000000   1.000000
max     67.100000   2.420000   81.000000   1.000000

```

```
df.isnull()
```

```

      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin
BMI \
0      False      False      False      False      False
False
1      False      False      False      False      False
False
2      False      False      False      False      False
False
3      False      False      False      False      False
False
4      False      False      False      False      False
False
..      ...      ...      ...      ...      ...      ..
.
763     False     False      False      False      False
False
764     False     False      False      False      False
False
765     False     False      False      False      False
False
766     False     False      False      False      False
False
767     False     False      False      False      False
False

```

```

      Pedigree      Age      Outcome
0      False     False      False
1      False     False      False
2      False     False      False
3      False     False      False

```

4	False	False	False
...	...	...	...
763	False	False	False
764	False	False	False
765	False	False	False
766	False	False	False
767	False	False	False

[768 rows x 9 columns]

df.isnull().sum()

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
Pedigree	0
Age	0
Outcome	0

dtype: int64

df.dtypes

Pregnancies	int64
Glucose	int64
BloodPressure	int64
SkinThickness	int64
Insulin	int64
BMI	float64
Pedigree	float64
Age	int64
Outcome	int64

dtype: object

df.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 768 entries, 0 to 767

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	int64
3	SkinThickness	768 non-null	int64
4	Insulin	768 non-null	int64
5	BMI	768 non-null	float64
6	Pedigree	768 non-null	float64
7	Age	768 non-null	int64

```
8 Outcome 768 non-null int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
df.shape
```

```
(768, 9)
```

```
x =
df[['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
    'BMI', 'Pedigree', 'Age']]
```

```
x
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1
..	...	...	...	...	...	...
763	10	101	76	48	180	32.9
764	2	122	70	27	0	36.8
765	5	121	72	23	112	26.2
766	1	126	60	0	0	30.1
767	1	93	70	31	0	30.4

	Pedigree	Age
0	0.627	50
1	0.351	31
2	0.672	32
3	0.167	21
4	2.288	33
..	...	...
763	0.171	63
764	0.340	27
765	0.245	30
766	0.349	47
767	0.315	23

```
[768 rows x 8 columns]
```

```
y = df[['Outcome']]
```

```
y
```

	Outcome
0	1
1	0
2	1
3	0
4	1
..	...
763	0
764	0
765	0
766	1
767	0

```
[768 rows x 1 columns]
```

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test =  
train_test_split(x,y,test_size=0.8,random_state=10)
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
Kn = KNeighborsClassifier()
```

```
Kn.fit(x_train,y_train)
```

```
C:\Users\D-Comp-PLII-17\anaconda3\New folder\Lib\site-packages\  
sklearn\neighbors\_classification.py:228: DataConversionWarning: A  
column-vector y was passed when a 1d array was expected. Please change  
the shape of y to (n_samples,), for example using ravel().  
    return self._fit(X, y)
```

```
KNeighborsClassifier()
```

```
y_prd = Kn.predict(x_test)
```

```
y_prd
```

```
array([1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1,  
0,  
0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,  
1,  
0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,  
0,  
0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0,
```

```
0,
1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0,
0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1,
1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1,
0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0,
1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1,
0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0,
1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0,
0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0,
1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0,
0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0,
0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0,
0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0,
0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0,
1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0,
1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0,
0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0,
0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0,
0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1,
0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0],
dtype=int64)
```

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
print(classification_report(y_prd, y_test))
```

	precision	recall	f1-score	support
0	0.79	0.77	0.78	411
1	0.56	0.58	0.57	204
accuracy			0.71	615
macro avg	0.67	0.68	0.67	615
weighted avg	0.71	0.71	0.71	615

```
cm = confusion_matrix(y_prd, y_test)
```

```
cm
```

```
array([[316, 95],  
       [ 85, 119]], dtype=int64)
```

```
a = accuracy_score(y_prd, y_test)
```

```
print(f"Accuracy:{a}")
```

```
Accuracy:0.7073170731707317
```